

StockPulse — Scaling & Monitoring Reference Guide

Covers: Horizontal Scaling · Vertical Scaling · HPA · Prometheus · Grafana

Project: StockPulse (Next.js on Kubernetes)

Date: June 2026

Covers: Horizontal Scaling · Vertical Scaling · HPA · Prometheus · Grafana

Project: StockPulse (Next.js on Kubernetes)

Date: June 2026

■ Table of Contents

- › [Concepts Index](#concepts-index)
- › [Vertical Scaling](#vertical-scaling)
- › [Horizontal Scaling (Manual)](#horizontal-scaling-manual)
- › [Horizontal Pod Autoscaler (HPA)](#horizontal-pod-autoscaler-hpa)
- › [Observability: Prometheus + Grafana](#observability-prometheus--grafana)
- › [Generating Real Test Traffic & Errors](#generating-real-test-traffic--errors)
- › [All Commands Quick Reference](#all-commands-quick-reference)
- › [Files Created in This Session](#files-created-in-this-session)
- › [Key Lessons Learned](#key-lessons-learned)

■ ■ Concepts Index

Concept	Section	Plain English Meaning
Vertical Scaling	§2	Give your existing container MORE CPU and RAM
Horizontal Scaling	§3	Spin up MORE identical containers
Resource Request	§2	Minimum CPU/RAM Kubernetes guarantees to a pod
Resource Limit	§2	Maximum CPU/RAM a pod is allowed to use before being killed
OOMKilled	§2	Pod was killed because it exceeded its memory limit
HPA	§4	Robot that automatically adds/removes pods based on CPU usage
Metrics Server	§4	K8s add-on that collects live CPU/RAM stats from all pods
kube-state-metrics	§5	Reports Kubernetes-level stats (pod counts, restart counts)
Node Exporter	§5	Reports host machine stats (machine CPU, disk, network)
Prometheus	§5	Time-series database that scrapes and stores metrics from pods
Scraping	§5	Prometheus pulling metrics from a pod every 15 seconds
PromQL	§5	Prometheus Query Language — used to ask questions about metrics
Grafana	§5	Visual dashboard tool that connects to Prometheus and draws graphs
up metric	§5	A Prometheus metric — 1 means the target is healthy, 0 means down
ServerSideApply	§5	ArgoCD option that fixes the "CRD annotations too long" sync error
kube-prometheus-stack	§5	The official community Helm chart that installs Prometheus + Grafana together
Time-series	§5	Data recorded over time — e.g., CPU at 12:00=12%, at 12:01=15%

■ ■ Vertical Scaling

What it is

Give a single pod more CPU cores and more RAM to handle heavier workloads.

When to use it

- Your app is CPU-bound (e.g., complex calculations, image processing)
- Your app cannot be made stateless (e.g., in-memory session data that can't be shared)
- Quick fix when you don't have time to refactor for horizontal scaling

Analogy

You give your one delivery driver a bigger, faster motorcycle. Simple. But eventually there's a maximum motorcycle size — and if that driver gets sick, all deliveries stop.

How to configure in this project

Configured in `helm/stock-pulse/values.yaml` under the `resources` key:

```
resources:
  requests:
    cpu: 10m      # Guaranteed minimum: 0.01 CPU cores
    memory: 128Mi # Guaranteed minimum: 128 MB RAM
  limits:
    cpu: 500m     # Hard max: 0.5 CPU cores
    memory: 512Mi # Hard max: 512 MB RAM. Exceed this = OOMKilled!
```

Understanding the CPU units

Value	Meaning
1000m	1 full CPU core
500m	0.5 CPU cores (half a core)
250m	0.25 CPU cores (quarter core)
10m	0.01 CPU cores (almost nothing — good for idle apps)

Pros & Cons

■ Pros	■ Cons
Simple — just change two numbers	Physical CPU/RAM ceiling exists
No code changes required	Single point of failure

■ Pros	■ Cons
Instant effect	Usually requires downtime on real servers

↔ ■ Horizontal Scaling (Manual)

What it is

Run multiple identical pods simultaneously. The Kubernetes Service load balances traffic across all of them.

When to use it

- Your app is stateless (each request is self-contained — no shared in-memory state)
- You expect traffic spikes and need high availability
- You want zero downtime if one pod crashes

Analogy

You hire 5 delivery drivers, each with their own standard motorcycle. If one driver calls in sick, the other 4 keep working. As you get more orders, you hire more drivers.

How to configure in this project

In `helm/stock-pulse/values.yaml`:

```
replicaCount: 5 # Run 5 identical pods simultaneously
```

Push to GitHub → ArgoCD applies the change → Kubernetes scales automatically.

Commands

```
# Check how many pods are running right now
kubectl get pods -l app.kubernetes.io/name=stock-pulse

# Manually scale without editing values.yaml (temporary, ArgoCD will revert it)
kubectl scale rollout stock-pulse --replicas=3

# Watch pods scale up/down in real time
kubectl get pods -l app.kubernetes.io/name=stock-pulse --watch
```

Pros & Cons

■ Pros	■ Cons
No physical ceiling	App must be stateless
High availability (fault tolerant)	More complex architecture
Zero downtime scaling	More infrastructure cost
Works with Kubernetes load balancing	

■ Horizontal Pod Autoscaler (HPA)

What it is

The HPA is a Kubernetes controller that **automatically** adjusts the number of running pods based on real-time resource metrics (CPU, memory).

You set a **minimum**, a **maximum**, and a **target threshold**. The HPA watches metrics continuously and acts like a thermostat: when CPU gets too hot, it adds more pods; when it cools down, it removes them.

How it works — step by step

- › Metrics Server collects CPU usage data from all pods every 30 seconds.
- › The HPA controller queries the Metrics Server and calculates average CPU usage.
- › If average CPU usage > **targetCPUUtilizationPercentage**, it increases **replicas**.
- › Calculation: **desiredReplicas = ceil(currentReplicas * (currentCPU / targetCPU))**
- › After traffic drops, the HPA waits a **5-minute cooldown period** before scaling down.

Install Metrics Server (required for HPA)

```
minikube addons enable metrics-server
```

Configuration in `helm/stock-pulse/values.yaml`

```
autoscaling:
  enabled: true
  minReplicas: 2      # Never go below 2 pods (for high availability)
  maxReplicas: 10     # Never go above 10 pods
  targetCPUUtilizationPercentage: 50 # Scale up if average CPU > 50%
```

The HPA Template (`helm/stock-pulse/templates/hpa.yaml`)

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stock-pulse
spec:
  scaleTargetRef:
    apiVersion: argoproj.io/v1alpha1
    kind: Rollout      # Points to Argo Rollout, not a standard Deployment
    name: stock-pulse
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50

```

Commands

```

# Watch the HPA check CPU every few seconds
kubectl get hpa --watch

# Expected output before load:
# NAME           TARGETS   MINPODS   MAXPODS   REPLICAS
# stock-pulse    0%/50%    2         10        2

# Generate load to trigger auto-scale
kubectl run load-generator --image=busybox --restart=Never -- sh -c \
  "while true; do wget -q -O- http://stock-pulse-active:3000/api/whoami; done"

# Expected output during heavy load:
# NAME           TARGETS   MINPODS   MAXPODS   REPLICAS
# stock-pulse    78%/50%    2         10        5 ← automatically scaled up!

# Clean up load generator
kubectl delete pod load-generator

# After ~5 min cooldown, HPA scales back down:
# NAME           TARGETS   MINPODS   MAXPODS   REPLICAS
# stock-pulse    0%/50%    2         10        2 ← automatically scaled down!

```

Vertical vs Horizontal vs HPA Comparison

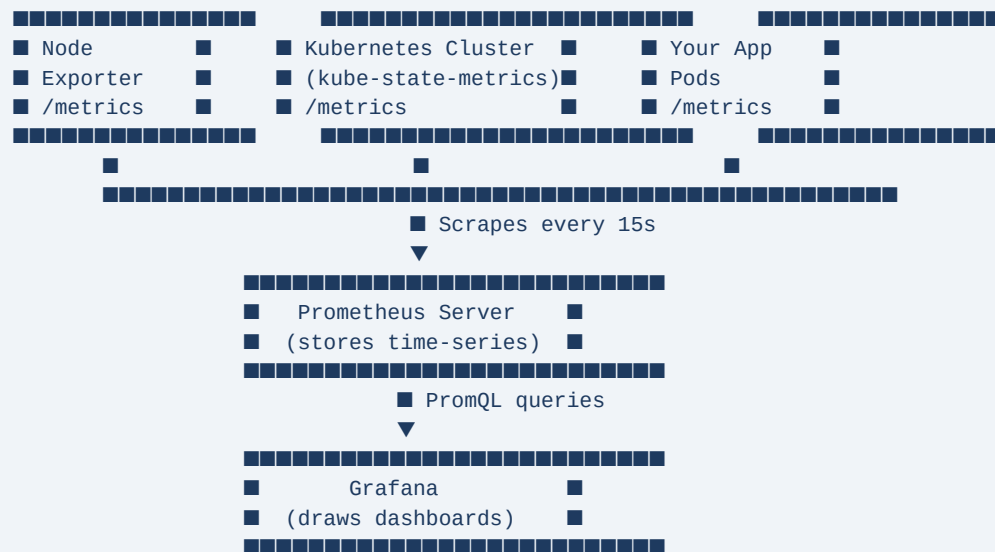
Feature	Vertical	Horizontal (Manual)	HPA (Auto)
How	Bigger CPU/RAM	More pods	Auto pod count
Requires code change?	No	No	No
Fault tolerant?	■	■	■

Feature	Vertical	Horizontal (Manual)	HPA (Auto)
Has ceiling?	■ (hardware)	■	■ (up to maxReplicas)
Responds to traffic?	■ Manual	■ Manual	■ Automatic
Best for	Dev/simple apps	Stateless apps	Production traffic spikes

■ Observability: Prometheus + Grafana

Architecture

Every 15 seconds, Prometheus visits each pod and asks for stats:



Components installed by kube-prometheus-stack

Component	Purpose
Prometheus Server	Scrapes and stores all metrics
Grafana	Web dashboard UI — connects to Prometheus
Node Exporter	Collects host machine CPU, RAM, disk, network
kube-state-metrics	Collects Kubernetes object stats (pod counts, restarts, etc.)
Alertmanager	*(disabled in dev)* Sends alerts via email/Slack on rules

ArgoCD Application (argocd/monitoring-stack.yaml)

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: monitoring-stack
  namespace: argocd
spec:
  project: default
  source:
    chart: kube-prometheus-stack
    repoURL: https://prometheus-community.github.io/helm-charts
    targetRevision: 61.3.0
    helm:
      values: |
        grafana:
          adminPassword: "admin"
          persistence:
            enabled: false
        alertmanager:
          enabled: false
  destination:
    server: https://kubernetes.default.svc
    namespace: monitoring
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
      - ServerSideApply=true # CRITICAL: Fixes "CRD annotations too long" error
      - Replace=true # CRITICAL: Ensures CRDs install before resources
```

Why ServerSideApply=true?

The Prometheus CRD files are hundreds of kilobytes. ArgoCD normally saves a copy of every resource inside a Kubernetes annotation, but annotations have a 262KB size limit. ServerSideApply tells ArgoCD to hand off the work to Kubernetes itself, bypassing this limit.

Deployment Commands


```
# Apply monitoring stack application to ArgoCD
kubectl apply -f argocd/monitoring-stack.yaml

# Watch monitoring pods come online (takes 2-3 min)
kubectl get pods -n monitoring --watch

# Access Grafana
kubectl port-forward svc/monitoring-stack-grafana -n monitoring 3002:80 &
# Open: http://localhost:3002 Login: admin / admin

# Access Prometheus directly
kubectl port-forward svc/monitoring-stack-kube-prom-prometheus -n monitoring 9090:9090 &
# Open: http://localhost:9090

# Verify Prometheus is scraping successfully
kubectl exec -n monitoring prometheus-monitoring-stack-kube-prom-prometheus-0 \
  -c prometheus -- wget -qO- "http://localhost:9090/api/v1/query?query=up"
```

Recommended Grafana Dashboards to Import

Go to: **Grafana** → **Dashboards** → **New** → **Import** → enter ID → **Load** → **Select Prometheus**

Dashboard ID	Name	What you see
6417	Kubernetes Cluster Overview	CPU, RAM, pod counts per namespace
14205	Kubernetes All-In-One	Detailed namespace + workload view
1860	Node Exporter Full	Host machine stats: CPU, memory, disk, network

Useful PromQL Queries (run in Prometheus at port 9090)

```
# Is each scrape target alive? (1 = up, 0 = down)
up

# Average CPU usage across all your app pods (%)
rate(container_cpu_usage_seconds_total{namespace="default",container="stock-pulse"}[5m])...

# Memory usage in MB for your app
container_memory_usage_bytes{namespace="default",container="stock-pulse"} / 1024 / 1024

# Total pod restart count in the default namespace
kube_pod_container_status_restarts_total{namespace="default"}

# HTTP request rate (if exposed by your app)
rate(http_requests_total[5m])
```

■ Generating Real Test Traffic & Errors

Use these commands to create visible activity on your Grafana dashboards.

Generate CPU Load (trigger HPA scale-up)

```
# Start a busybox pod inside the cluster that hammers your app
kubectl run load-generator --image=busybox --restart=Never -- sh -c \
  "while true; do wget -q -O- http://stock-pulse-active:3000/api/whoami; done"

# Stop it when done
kubectl delete pod load-generator
```

Deliberately Crash a Pod (see restart events in Grafana)

```
# Delete a pod – Kubernetes will automatically restart it
kubectl delete pod $(kubectl get pods -l app.kubernetes.io/name=stock-pulse \
  -o jsonpath='{.items[0].metadata.name}')

# Then watch what happens:
kubectl get pods -l app.kubernetes.io/name=stock-pulse --watch
# You will see: Running → Error → ContainerCreating → Running
```

Trigger Rate Limiting (generate 503 errors)

```
# Fire 200 rapid requests through the Ingress
MINIKUBE_IP=$(minikube ip)
for i in $(seq 1 200); do
  wget -q -O- -H "Host: stock-pulse.local" "http://$MINIKUBE_IP/"
done
```

What you see on the Grafana dashboard

What happened	Where to see it in Grafana
Load generator running	CPU Usage graph spikes up
Pod was deleted	Pod Restart Count increases by 1
HPA scaled from 2 to 7 pods	Pod Count graph jumps
Rate limiter blocked requests	Network throughput drops (blocked 503s return small payloads)

■ All Commands Quick Reference

Scaling

```
# Check HPA status
kubectl get hpa

# Watch HPA changes live
kubectl get hpa --watch

# Manually scale (temporary, ArgoCD will revert)
kubectl scale rollout stock-pulse --replicas=5

# Check resource usage of running pods
kubectl top pods -l app.kubernetes.io/name=stock-pulse

# Check resource usage of nodes
kubectl top nodes
```

Monitoring

```
# Get all monitoring pods (check they are all Running)
kubectl get pods -n monitoring

# Get all monitoring services (get exact service names for port-forward)
kubectl get svc -n monitoring

# Port-forward Grafana to localhost:3002
kubectl port-forward svc/monitoring-stack-grafana -n monitoring 3002:80 &

# Port-forward Prometheus to localhost:9090
kubectl port-forward svc/monitoring-stack-kube-prom-prometheus -n monitoring 9090:9090 &

# Restart monitoring stack if it gets stuck
kubectl rollout restart deployment -n monitoring

# Check Prometheus targets (which pods it is scraping)
# Open in browser: http://localhost:9090/targets
```

Debugging

```
# Check pod resource limits/requests
kubectl describe pod <pod-name> | grep -A 5 Limits

# Check HPA events (useful when it fails to scale)
kubectl describe hpa stock-pulse

# Check if Metrics Server is working
kubectl top pods

# Check events in the monitoring namespace
kubectl get events -n monitoring --sort-by='.lastTimestamp'
```

■ Files Created in This Session

File	Purpose
helm/stock-pulse/templates/hpa.yaml	HPA definition — auto-scales pods based on CPU
argocd/monitoring-stack.yaml	Deploys Prometheus + Grafana via ArgoCD
start-system.sh (updated)	Now also starts Grafana and Prometheus port-forwards

helm/stock-pulse/values.yaml — key changes

```
# Before this session:
replicaCount: 1
autoscaling:
  enabled: false
  minReplicas: 1

# After this session:
replicaCount: 5
autoscaling:
  enabled: true
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilizationPercentage: 50
```

■ Key Lessons Learned

- › **HPA needs scaleTargetRef to point to Rollout, not Deployment** — since we replaced our standard Deployment with an Argo Rollout for Blue-Green deployments, the HPA must reference the `argoproj.io/v1alpha1 Rollout` kind, not `apps/v1 Deployment`.
- › **Metrics Server is not installed by default in Minikube** — The HPA does nothing without it. Always run `minikube addons enable metrics-server` before expecting HPA to work.
- › **HPA has a 5-minute cooldown before scaling down** — This is intentional to prevent rapid oscillation (scale up → down → up within seconds). After load stops, wait at least 5 minutes to see the pods scale back to `minReplicas`.
- › **ServerSideApply=true is mandatory for kube-prometheus-stack** — The Prometheus CRDs are enormous. Without this ArgoCD sync option, ArgoCD will always fail with "metadata.annotations: Too long: may not be more than 262144 bytes".
- › **The "CRDs must be installed first" error is fixed by Replace=true** — ArgoCD tries to apply all resources simultaneously. `Replace=true` tells it to use `kubectl replace` for CRDs, which installs them cleanly before

dependent resources are applied.

- › **Prometheus uses a PULL model** — Unlike most monitoring tools, Prometheus comes to your pods and asks for metrics (scraping). Your pods don't push data. This means if a pod is dead, Prometheus immediately knows because there is no response at the `/metrics` endpoint.
- › **`kubectl top pods` only works if Metrics Server is running** — If you see `error: metrics not available yet`, the Metrics Server hasn't finished initializing. Wait 60 seconds and try again.
- › **Watching `kubectl get pods --watch` shows the full pod lifecycle** — You can literally see a pod go from **Running** → **Error** → **ContainerCreating** → **Running** in real time when you delete it. This is Kubernetes self-healing in action.